



資料結構

Data Structure

Lab

姓名： 黃聖傑

學號： 114310140

Lab01-Q1. (題目編號)

write a function that receives the layer number and returns the sum of all nodes at that layer.

Code

```
#include <iostream>
#include <queue>
#include <vector>
#include <utility>
using namespace std;
const int EMPTY = 0; // 用 0 代表沒有節點
class TreeNode {
public:
    int value; // 節點值
    TreeNode* left; // 左子節點
    TreeNode* right; // 右子節點

    TreeNode(int val) : value(val), left(nullptr), right(nullptr) {} // 初始化節點
};

class BinaryTree {
public:
    TreeNode* root; // 樹根節點

    BinaryTree() : root(nullptr) {}

    TreeNode* buildTree(const vector<int>& arr) { // 建立樹
        if (arr.empty() || arr[0] == EMPTY) return nullptr;

        queue<TreeNode*> q; // 建立 queue 儲存待處理的節點指標
        root = new TreeNode(arr[0]); // 建立根節點 (陣列第一個元素)
        q.push(&root); // 將根節點的指標加入 queue

        size_t i = 1; // 陣列索引
        while (!q.empty() && i < arr.size()) {
            TreeNode** nodePtr = q.front(); // 取出 queue 的第一個節點指標
            q.pop(); // 將該節點從 queue 中刪除
```

```

        // 左子節點
        if (i < arr.size()) {
            if (arr[i] != EMPTY) {
                (*nodePtr)->left = new TreeNode(arr[i]); // 添加左子節點
                q.push(&((*nodePtr)->left)); // 將左子節點的指標加入 queue
            }
            i++;
        }
        // 右子節點
        if (i < arr.size()) {
            if (arr[i] != EMPTY) {
                (*nodePtr)->right = new TreeNode(arr[i]); // 添加右子節點
                q.push(&((*nodePtr)->right)); // 將右子節點的指標加入 queue
            }
            i++;
        }
    }
    return root;
}

void Depth_first_search(TreeNode* node) { // 深度優先搜尋
    if (node == nullptr) return;
    cout << node->value << " "; // 訪問當前節點的值
    Depth_first_search(node->left); // 遞迴遍歷左子樹
    Depth_first_search(node->right); // 遞迴遍歷右子樹
}

void Breadth_first_search(TreeNode* root) {
    if (root == nullptr) return;
    queue<TreeNode*> q; // 建立 queue 儲存待處理的節點指標
    q.push(root); // 將根節點的指標加入 queue

    while (!q.empty()) {
        TreeNode* current = q.front(); // 取出 queue 的第一個節點指標
        q.pop(); // 將該節點從 queue 中刪除
        cout << current->value << " ";
        if (current->left) q.push(current->left); // 將左子節點的指標加入
queue
    }
}

```

```

        if (current->right) q.push(current->right); // 將右子節點的指標加入
queue
    }
}

int getHeight(TreeNode* root) { // 計算樹高
    if (root == nullptr) return 0; // 空樹高度為 0
    int leftHeight = getHeight(root->left); // 左子樹高度
    int rightHeight = getHeight(root->right); // 右子樹高度
    return max(leftHeight, rightHeight) + 1; // 返回較大高度 + 1 (包含當前節
點)
}

int sumAtLayer(TreeNode* root, int layer) {
    if (layer <= 0) {
        return 0; // 保護：無效層號直接返回 0
    }
    if (root == nullptr) return 0; // 空樹返回 0
    if (layer > getHeight(root)) { // 超過樹高顯示訊息並返回 0
        cout << "The layer exceeds the tree height." << endl;
        return 0;
    }
    queue<pair<TreeNode*, int>> q;
    q.push({root, 1});
    int sum = 0;

    while (!q.empty()) {
        TreeNode* node;
        int level;
        tie(node, level) = q.front();
        q.pop();

        if (level == layer) { // 當前節點在目標層
            sum += node->value; // 累加節點值
        }
        if (level < layer) { // 只有當前層小於目標層才繼續添加子節點
            if (node->left) q.push({node->left, level + 1}); // 將左子節點和
層號加入 queue
            if (node->right) q.push({node->right, level + 1}); // 將右子節點
和層號加入 queue

```

```

        }
    }
    cout << "Sum at layer " << layer-1 << ": " << sum << endl;
    return sum;
}

};

int main() {
    BinaryTree tree;
    vector<int> arr = { 1, 2, 3, 4, 5, 6, 7, 8, 9, EMPTY, EMPTY, 10, 11, EMPTY,
EMPTY };
    tree.buildTree(arr);

    cout << "DFS Result: ";
    tree.Depth_first_search(tree.root);
    cout << endl;

    cout << "BFS Result: ";
    tree.Breadth_first_search(tree.root);
    cout << endl;

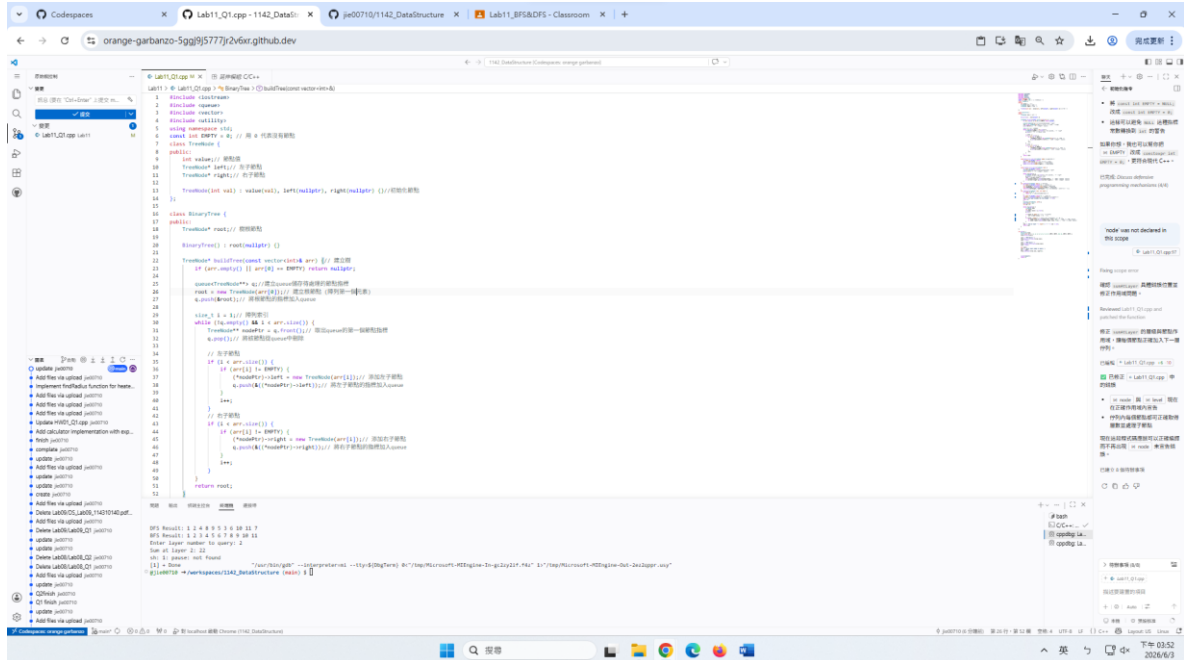
    int layer;
    cout << "Enter layer number to query: ";
    cin >> layer;
    tree.sumAtLayer(tree.root, layer+1);

    system("pause");
    return 0;
}

```

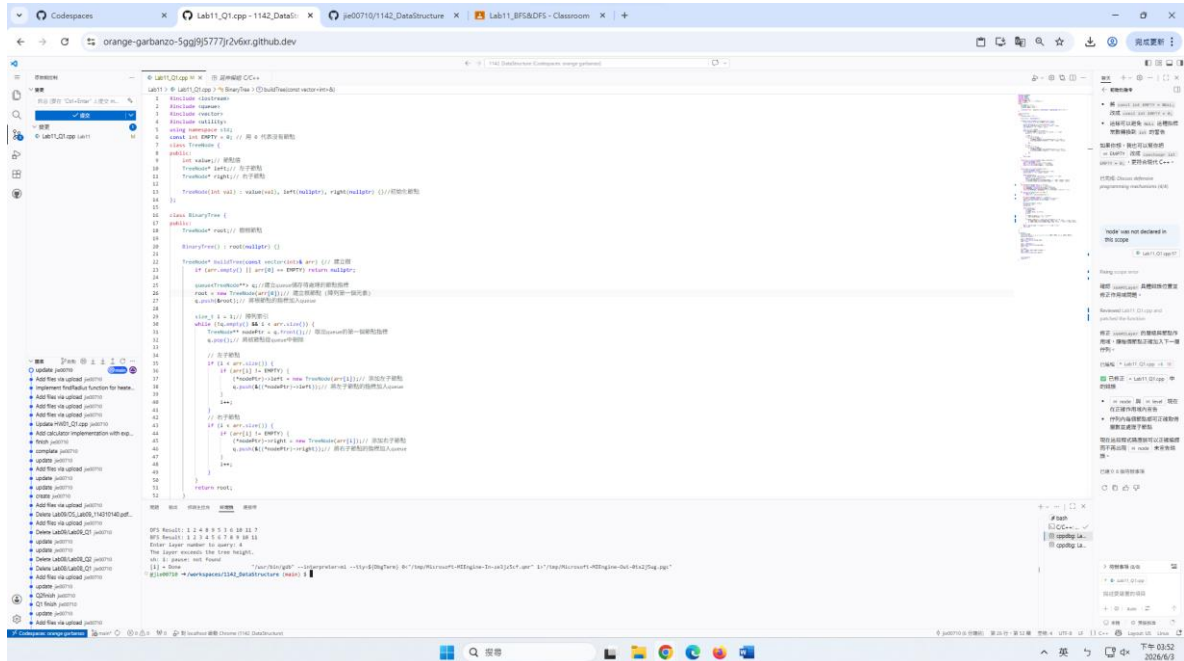
Result

```
DFS Result: 1 2 4 8 9 5 3 6 10 11 7
BFS Result: 1 2 3 4 5 6 7 8 9 10 11
Enter layer number to query: 2
Sum at layer 2: 22
```



```
Lab11_Q1.cpp - 2023/06/03 14:00:00
1 // Lab11_Q1.cpp : 此文件包含 C++ 应用程序的入口点。
2 #include <iostream>
3 #include <vector>
4 #include <queue>
5 using namespace std;
6 struct TreeNode {
7     int val;
8     TreeNode* left;
9     TreeNode* right;
10 };
11
12 // 递归函数
13 void dfs(TreeNode* root, int val, int sum, int layer) {
14     if (root == nullptr) return;
15     if (layer == 0) {
16         sum += root->val;
17         return;
18     }
19     dfs(root->left, val, sum, layer-1);
20     dfs(root->right, val, sum, layer-1);
21 }
22
23 // 广度优先遍历
24 void bfs(TreeNode* root, int val, int sum, int layer) {
25     if (root == nullptr) return;
26     queue<TreeNode*> q;
27     q.push(root);
28     while (!q.empty()) {
29         int size = q.size();
30         for (int i = 0; i < size; i++) {
31             TreeNode* node = q.front();
32             q.pop();
33             if (layer == 0) {
34                 sum += node->val;
35             }
36             if (node->left != nullptr) q.push(node->left);
37             if (node->right != nullptr) q.push(node->right);
38         }
39     }
40 }
41
42 // 主函数
43 int main() {
44     // 创建二叉树
45     TreeNode* root = new TreeNode(1);
46     root->left = new TreeNode(2);
47     root->right = new TreeNode(3);
48     root->left->left = new TreeNode(4);
49     root->left->right = new TreeNode(5);
50     root->right->left = new TreeNode(6);
51     root->right->right = new TreeNode(7);
52     root->left->left->left = new TreeNode(8);
53     root->left->left->right = new TreeNode(9);
54     root->left->right->right = new TreeNode(10);
55     root->right->right->right = new TreeNode(11);
56
57     // 测试 DFS
58     int sum = 0;
59     dfs(root, 0, sum, 2);
60     cout << "DFS Result: " << sum << endl;
61
62     // 测试 BFS
63     sum = 0;
64     bfs(root, 0, sum, 2);
65     cout << "BFS Result: " << sum << endl;
66
67     // 测试查询层数
68     int layer = 2;
69     sum = 0;
70     bfs(root, 0, sum, layer);
71     cout << "Enter layer number to query: " << layer << endl;
72     cout << "Sum at layer " << layer << ": " << sum << endl;
73
74     return 0;
75 }
```

```
DFS Result: 1 2 4 8 9 5 3 6 10 11 7
BFS Result: 1 2 3 4 5 6 7 8 9 10 11
Enter layer number to query: 4
The layer exceeds the tree height.
```



```
Lab11_Q1.cpp - 2023/06/03 14:00:00
1 // Lab11_Q1.cpp : 此文件包含 C++ 应用程序的入口点。
2 #include <iostream>
3 #include <vector>
4 #include <queue>
5 using namespace std;
6 struct TreeNode {
7     int val;
8     TreeNode* left;
9     TreeNode* right;
10 };
11
12 // 递归函数
13 void dfs(TreeNode* root, int val, int sum, int layer) {
14     if (root == nullptr) return;
15     if (layer == 0) {
16         sum += root->val;
17         return;
18     }
19     dfs(root->left, val, sum, layer-1);
20     dfs(root->right, val, sum, layer-1);
21 }
22
23 // 广度优先遍历
24 void bfs(TreeNode* root, int val, int sum, int layer) {
25     if (root == nullptr) return;
26     queue<TreeNode*> q;
27     q.push(root);
28     while (!q.empty()) {
29         int size = q.size();
30         for (int i = 0; i < size; i++) {
31             TreeNode* node = q.front();
32             q.pop();
33             if (layer == 0) {
34                 sum += node->val;
35             }
36             if (node->left != nullptr) q.push(node->left);
37             if (node->right != nullptr) q.push(node->right);
38         }
39     }
40 }
41
42 // 主函数
43 int main() {
44     // 创建二叉树
45     TreeNode* root = new TreeNode(1);
46     root->left = new TreeNode(2);
47     root->right = new TreeNode(3);
48     root->left->left = new TreeNode(4);
49     root->left->right = new TreeNode(5);
50     root->right->left = new TreeNode(6);
51     root->right->right = new TreeNode(7);
52     root->left->left->left = new TreeNode(8);
53     root->left->left->right = new TreeNode(9);
54     root->left->right->right = new TreeNode(10);
55     root->right->right->right = new TreeNode(11);
56
57     // 测试 DFS
58     int sum = 0;
59     dfs(root, 0, sum, 2);
60     cout << "DFS Result: " << sum << endl;
61
62     // 测试 BFS
63     sum = 0;
64     bfs(root, 0, sum, 2);
65     cout << "BFS Result: " << sum << endl;
66
67     // 测试查询层数
68     int layer = 4;
69     sum = 0;
70     bfs(root, 0, sum, layer);
71     cout << "Enter layer number to query: " << layer << endl;
72     cout << "The layer exceeds the tree height." << endl;
73
74     return 0;
75 }
```